

API 与 SQL 耗时监控方案文档

架构总览

本项目实现了一套通用的请求/SQL 耗时监控系统，核心思路是**拦截层模式**：



所有 SQL 监控最终都落到 ADO.NET 层。无论 ORM 如何封装，底层都是通过 `DbConnection.CreateCommand()` 创建 `DbCommand`，再调用 `ExecuteReader` / `ExecuteNonQuery` / `ExecuteScalar`。因此，在 **ADO.NET 层做拦截是通用方案**。

一、API 接口耗时统计

原理

通过 ASP.NET Core **Middleware** 在请求管道最外层包裹 `Stopwatch`，记录每个 HTTP 请求的：

- 请求方法 (GET/POST/...)
- 请求路径
- 响应状态码
- 耗时 (毫秒)

核心代码

Infrastructure/ApiTimingMiddleware.cs :

```
public class ApiTimingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ApiTimingMiddleware> _logger;

    public ApiTimingMiddleware(RequestDelegate next, ILogger<ApiTimingMiddleware>
logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var sw = Stopwatch.StartNew();
```

```

        await _next(context); // 执行后续管道

        sw.Stop();

        var metric = new ApiMetric
        {
            Method      = context.Request.Method,
            Path         = context.Request.Path,
            StatusCode   = context.Response.StatusCode,
            ElapsedMs    = sw.ElapsedMilliseconds,
            Timestamp    = DateTime.UtcNow
        };

        // API 是否"异常"由状态码决定: >= 400 视为异常
        _logger.LogInformation("[API] {Method} {Path} => {StatusCode} in {Elapsed}ms",
            metric.Method, metric.Path, metric.StatusCode, metric.ElapsedMs);

        ApiMetricStore.Add(metric);
    }
}

```

注册顺序（Program.cs）—— 必须在管道最前面：

```

app.UseMiddleware<ApiTimingMiddleware>();
app.MapControllers();

```

查看指标

端点	说明
GET /api/metrics/api	查看所有 API 耗时记录
DELETE /api/metrics	清空所有指标

示例输出

```

{
  "method": "GET",
  "path": "/api/users/efcore",
  "statusCode": 200,
  "elapsedMs": 42,
  "success": true,
  "timestamp": "2026-05-08T10:00:00Z"
}

```

二、ORM 查询耗时统计

本项目支持三种 ORM + 纯 SQL 的监控，每种拦截机制不同，但目标一致：记录 SQL 语句、耗时、成功/失败。

2.1 EF Core — IDbCommandInterceptor

EF Core 提供了原生的拦截器机制，无需包装连接。

```

public class EfCoreTimingInterceptor : DbCommandInterceptor
{
    // 查询返回结果集后触发
    public override DbDataReader ReaderExecuted(
        DbCommand command, CommandExecutedEventData eventData, DbDataReader result)
    {
        // eventData.Duration 就是 EF Core 内部计时的耗时
        LogSuccess(command, eventData.Duration.TotalMilliseconds, "Reader");
        return result;
    }

    // ExecuteNonQuery (INSERT/UPDATE/DELETE) 后触发
    public override int NonQueryExecuted(
        DbCommand command, CommandExecutedEventData eventData, int result)
    {
        LogSuccess(command, eventData.Duration.TotalMilliseconds, "NonQuery", result);
        return result;
    }

    // ExecuteScalar 后触发
    public override object ScalarExecuted(
        DbCommand command, CommandExecutedEventData eventData, object? result)
    {
        LogSuccess(command, eventData.Duration.TotalMilliseconds, "Scalar");
        return result!;
    }

    // SQL 执行失败时触发
    public override void CommandFailed(DbCommand command, CommandErrorEventData
eventData)
    {
        LogFailure(command, eventData.Duration.TotalMilliseconds, eventData.Exception);
    }
}

```

注册方式:

```

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(connectionString)
        .AddInterceptors(new EfCoreTimingInterceptor(logger)));

```

优点: EF Core 原生支持, 精确计时 (eventData.Duration 包含了命令执行和数据读取的完整时间)。**适用场景:** 项目中使用 EF Core 的部分。

2.2 Dapper / Raw SQL — ADO.NET 装饰器

Dapper 和纯 SQL 字符串操作都通过 IDbConnection 执行 SQL。通过包装 DbConnection 和 DbCommand, 可以透明地拦截所有 SQL 执行。

核心: TimedDbConnection

```

public class TimedDbConnection : DbConnection
{
    private readonly DbConnection _inner; // 真正的连接 (如 SqliteConnection)
    private readonly ILogger _logger;
}

```

```

        private readonly string? _source;           // 标记来源："Dapper" 或 "RawSQL"

        protected override DbCommand CreateDbCommand()
            => new TimedDbCommand(_inner.CreateCommand(), _logger, _source);

        // 其余成员全部委托给 _inner
    }

```

核心：TimedDbCommand

```

public class TimedDbCommand : DbCommand
{
    private readonly DbCommand _inner; // 真正的命令
    private readonly ILogger _logger;
    private readonly string? _source;

    protected override DbDataReader ExecuteDbDataReader(CommandBehavior behavior)
    {
        var sw = Stopwatch.StartNew();
        try
        {
            var reader = _inner.ExecuteReader(behavior);
            sw.Stop();
            LogSuccess("ExecuteReader", sw.ElapsedMilliseconds);
            return reader;
        }
        catch (Exception ex)
        {
            sw.Stop();
            LogFailure(ex, sw.ElapsedMilliseconds);
            throw;
        }
    }

    // ExecuteNonQuery、ExecuteScalar 同理...
}

```

使用方式

```

// 创建带计时的连接
private DbConnection CreateTimedConnection(string? source = null)
{
    var inner = new SqliteConnection(_connectionString);
    return new TimedDbConnection(inner, _logger, source);
}

```

Dapper 调用：

```

public async Task<IEnumerable<User>> GetUsersViaDapper()
{
    var conn = CreateTimedConnection("Dapper");
    await using var _ = conn.ConfigureAwait(false);
    await conn.OpenAsync();
    // Dapper 通过 conn 执行 SQL，自动经过 TimedDbCommand
}

```

```

    }
    return await conn.QueryAsync<User>("SELECT * FROM Users");
}

```

纯 SQL 调用：

```

public async Task<IEnumerable<User>> GetUsersViaRawSql(string? whereClause = null)
{
    var conn = CreateTimedConnection("RawSQL");
    await using var _ = conn.ConfigureAwait(false);
    await conn.OpenAsync();

    var sql = "SELECT * FROM Users";
    if (!string.IsNullOrEmpty(whereClause))
        sql += $" WHERE {whereClause}";

    // 同样通过 conn 执行，自动经过 TimedDbCommand
    return await conn.QueryAsync<User>(sql);
}

```

关键点：Dapper 的 `QueryAsync` 本质上就是调用 `IDbConnection.CreateCommand() + ExecuteReader()`，所以使用 `TimedDbConnection` 包装后，Dapper 和纯 SQL 都能自动获得耗时监控，无需修改 Dapper 的调用方式。

2.3 SqlSugar — 内置 AOP

SqlSugar 自带 AOP 切面机制，无需包装连接。

```

private SqlSugarClient CreateSqlSugarClient()
{
    var db = new SqlSugarClient(new ConnectionConfig
    {
        ConnectionString = _connectionString,
        DbType = DbType.Sqlite,
        IsAutoCloseConnection = true,
        InitKeyType = InitKeyType.Attribute
    });

    // SQL 执行前
    db.Aop.OnLogExecuting = (sql, pars) =>
    {
        _logger.LogInformation("[SqlSugar] Executing | {Sql}", sql);
    };

    // SQL 执行后
    db.Aop.OnLogExecuted = (sql, pars) =>
    {
        _logger.LogInformation("[SqlSugar] Executed | {Sql}", sql);
    };

    // SQL 执行失败
    db.Aop.OnError = (ex) =>
    {
        _logger.LogError(ex, "[SqlSugar] Error | {Sql}", ex.Sql);
        SqlMetricStore.Add(new SqlMetric
        {
            Sql = ex.Sql,

```

```

        Success = false,
        Error = ex.Message,
        Timestamp = DateTime.UtcNow,
        Source = "SqlSugar"
    });
};

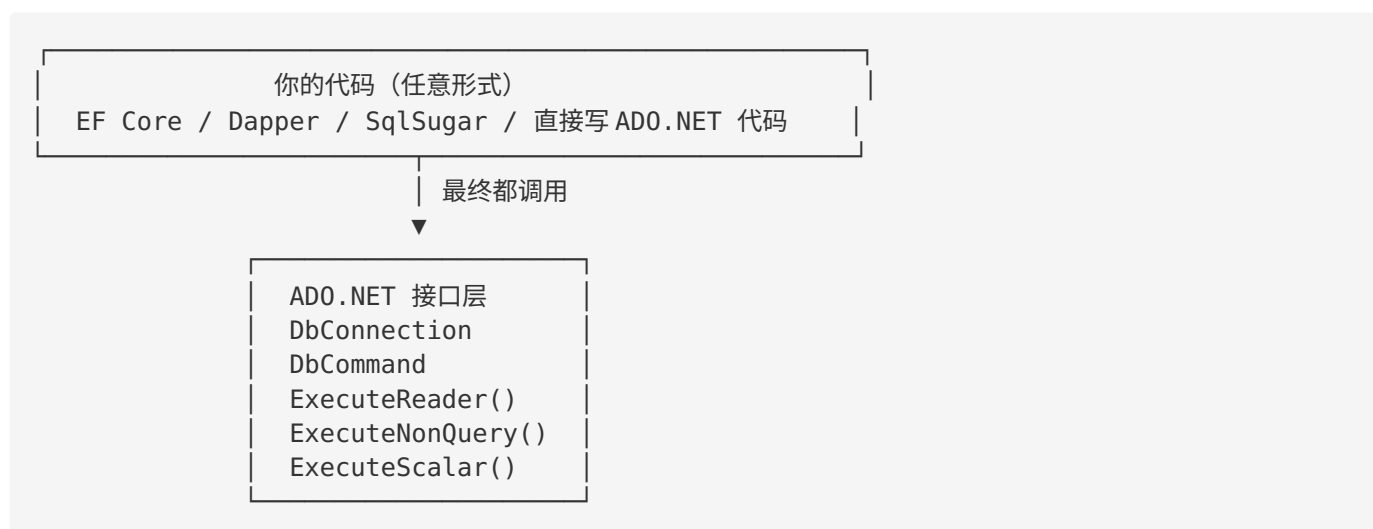
return db;
}

```

注意：SqlSugar 的 `OnLogExecuted` 没有直接提供耗时，如需精确计时，可以在 `OnLogExecuting` 中启动 `Stopwatch`，在 `OnLogExecuted` 中停止。

三、纯 SQL 字符串是否也是基于 ADO.NET?

是的。这是理解本方案的关键：



无论你是：

- EF Core 生成的 SQL
- Dapper 的 `connection.Query<T>("SELECT ...")`
- SqlSugar 的 `db.Queryable<T>()`
- 直接拼接 SQL 字符串：`"SELECT * FROM Users WHERE Name = '" + name + "'"`

它们最终都要通过 ADO.NET 的 `DbCommand` 执行。所以本方案中的 `TimedDbConnection` / `TimedDbCommand` 装饰器对所有场景都有效。

纯 SQL 示例

```

// 直接使用 ADO.NET + 拼接 SQL
var conn = CreateTimedConnection("RawSQL");
await conn.OpenAsync();

// 方式1：通过 Dapper 执行拼接 SQL
var users = await conn.QueryAsync<User>($"SELECT * FROM Users WHERE Name = '{name}'");

// 方式2：直接用 ADO.NET 执行
using var cmd = conn.CreateCommand(); // 返回的是 TimedDbCommand
cmd.CommandText = $"SELECT * FROM Users WHERE Name = '{name}'";

```

```
using var reader = await cmd.ExecuteReaderAsync();
// reader 自动经过 TimedDbCommand 的计时逻辑
```

两种方式都会被 `TimedDbCommand` 拦截并记录耗时。

四、指标存储与查询

内存存储

使用 `ConcurrentQueue` 实现线程安全的内存队列：

```
public static class SqlMetricStore
{
    private static readonly ConcurrentQueue<SqlMetric> _metrics = new();

    public static void Add(SqlMetric metric) => _metrics.Enqueue(metric);
    public static IEnumerable<SqlMetric> GetAll() => _metrics.ToArray();
    public static IEnumerable<SqlMetric> GetRecent(int count) =>
        _metrics.ToArray().TakeLast(count);
    public static void Clear() { while (_metrics.TryDequeue(out _)) { } }
}
```

查询端点

端点	方法	说明
/api/metrics/api	GET	所有 API 耗时记录
/api/metrics/sql	GET	所有 SQL 耗时记录
/api/metrics	DELETE	清空全部指标

日志输出

所有 SQL 计时结果同时写入 Serilog（控制台 + 文件）：

```
[10:00:01 INF] [Dapper] ExecuteReader | 12ms | SELECT * FROM Users
[10:00:01 INF] [EF Core] Reader | 8ms | SELECT "u"."Id", "u"."Name" FROM "Users" AS "u"
[10:00:01 INF] [SqlSugar] Executed | SELECT * FROM Users
[10:00:01 INF] [RawSQL] ExecuteReader | 5ms | SELECT * FROM Users WHERE Name = 'Alice'
```

五、扩展建议

方向	说明
持久化存储	将 <code>ConcurrentQueue</code> 替换为数据库表或时序数据库（InfluxDB）
OpenTelemetry	接入 <code>System.Diagnostics.Activity</code> 实现分布式追踪
告警	耗时超过阈值（如 500ms）时触发告警
慢 SQL 分析	按 <code>ElapsedMs</code> 排序，找出 Top N 慢查询
按接口聚合	将 API 指标和 SQL 指标通过 <code>TraceId</code> 关联，分析某个接口触发了哪些 SQL